# Python Engineering - Class 05

Lists, Tuples and Trajectories | EX13 - Trajectory with Lists and Tuples

**Class objective:** Learn to store collections of states and understand mutability, references and trajectory snapshots.

This document is the base exercise guide. It is not a full solution to copy: it organizes the problem, implementation steps, reference values and required validation.

## Path

```
01_python_bootcamp/
`-- week_02_data_structures/class_05_lists_tuples/
    |-- notes/
    `-- exercices/
```

## Required workflow

```
PROBLEM
INPUTS
OUTPUTS
EQUATIONS
SPECIAL CASES
DO I NEED ITERATION?
ALGORITHM
VALIDATION
```

Before coding, write these eight sections. The goal is to separate problem reasoning from syntax.

## EX13 - Trajectory with Lists and Tuples

**Problem:** Repeat vertical-motion simulation and store every state as a tuple (h,v,t) inside a list.

**Reference inputs:** h0=0, v0=50, t0=0, g=9.81, dt=0.1

**Equations / relationship:** Euler vertical motion; analytical validation

**Expected check:** trajectory is a list; each state is a tuple; max(state[0] for state in trajectory) matches stored maximum approximately.

### Implementation guide

- 1. Create h,v,t and trajectory lists.
- 2. Append initial state.
- 3. Iterate with Euler.
- 4. Append a new tuple each step.
- 5. Extract maximum/final data from trajectory.

### Before running

Predict sign, order of magnitude, data type and any relevant shape. Then compare the prediction with the actual output.

## Final validation

Answer these questions as comments at the end of the exercise. The goal is not memorized wording but demonstrating that you can explain the code flow.

- What is the difference between list and tuple?
- What does append() do?
- What does trajectory[-1] mean?
- Why does storing new tuples preserve previous states?
- What is aliasing?
- What does max(state[0] for state in trajectory) return?

## Completion criterion

The class is complete when the code runs, the reference checks make sense, you can explain every block without reading a solution, and you can modify at least one input while predicting the expected effect.